COMP 551 Assignment 4 Report

Hadi El Nawfal, Bora Sanal, Andrea Gómez

April 2026

1 Abstract

This report studies four-class news topic classification on the AG News dataset using two sequence modeling pipelines: a bidirectional LSTM trained from scratch and a fine-tuned pretrained Transformer (DistilBERT). For the LSTM, we implemented the full preprocessing pipeline ourselves, including word-level tokenization, vocabulary construction from the training split, truncation, padding, and length-aware sequence handling with `pack_padded_sequence`. We first trained a baseline bidirectional LSTM with last-hidden-state pooling, which achieved a validation accuracy of **91.75%** and a test accuracy of **91.39%**. We then performed an ablation study over recurrent architecture, directionality, and pooling, where the best LSTM variant was a bidirectional LSTM with max pooling, reaching **92.62%** validation accuracy and **91.84%** test accuracy. We also compared random initialization against GloVe initialization for this best LSTM setting, and GloVe improved test accuracy from **92.00%** to **93.17%**. For the Transformer, we fine-tuned `distilbert-base-uncased` and compared full fine-tuning, a frozen encoder, and LoRA. Full fine-tuning performed best, obtaining **94.80%** validation accuracy and **94.58%** test accuracy, clearly outperforming the LSTM-based models. The results show that pretrained Transformers provide the strongest accuracy, while the LSTM experiments remain competitive and provide useful insight into design choices such as pooling, bidirectionality, and embedding initialization.

2 Introduction

The AG News dataset is a standard benchmark for news topic classification. It contains 120,000 training examples and 7,600 test examples distributed across four balanced classes: World, Sports, Business, and Sci/Tech. Although the task is conceptually simple, the input consists of raw text sequences with variable lengths, so an effective modeling pipeline must decide how to tokenize text, represent words or subwords as integers, and handle padding during batching. These preprocessing choices are not minor implementation details; they directly affect what information the model receives and how efficiently it can learn from the data. In this report, we compare two sequence modeling paradigms for the same classification task. The first is a bidirectional LSTM trained from scratch, which requires building a word-level vocabulary, mapping tokens to IDs, truncating long sequences, and ensuring that padding tokens do not distort the final sequence representation. The second is a pretrained DistilBERT Transformer, which uses its own subword tokenizer, special tokens, and attention masks, and is then fine-tuned for classification. Beyond this baseline comparison, we extend the study on both sides. For the LSTM family, we examine the effect of architectural choices through an ablation study over directionality, pooling strategy, and recurrent unit type, and we also compare random embedding initialization against pretrained GloVe embeddings. For the Transformer family, we compare different adaptation strategies, namely full fine-tuning, frozen-encoder training, and LoRA, and we further study the effect of the maximum input length on the accuracy–efficiency trade-off. This broader comparison is important because the two approaches differ not only in architecture, but also in how they process text, how much prior linguistic knowledge they bring to the task, and how sensitive they are to practical design choices. Sequence modeling choices matter because they determine how well a model can represent context, rare words, and long-range dependencies. In the LSTM setting, the model must learn useful embeddings and sequence patterns entirely from AG News, making vocabulary design and padding handling especially important. In the Transformer setting, pretraining provides strong contextual representations from the start, but correct use of the pretrained tokenizer and masking mechanism is essential. Comparing these two workflows therefore shows both the practical and performance differences between training a model from scratch and adapting a pretrained language model.

3 Methods

3.1 Data Preparation

We loaded the dataset via HuggingFace's `load_dataset('ag_news')`. From the official 120K training set, we created a 90/10 stratified split (seed 2026), yielding 108,000 training and 12,000 validation examples. The 7,600-example test set was held out for final evaluation.

3.2 LSTM Pipeline

Tokenization & vocabulary. We use NLTK's `word_tokenize` with lowercasing, which splits text into words and separates punctuation as distinct tokens. The vocabulary was built from training data only, retaining words with frequency ≥ 3, resulting in a vocabulary of 39,286 words. Two special tokens are reserved: PAD (index 0) and UNK (index 1). Each text is converted to integer IDs and truncated to 128 tokens. Empty sequences receive a single UNK to prevent `pack_padded_sequence` crashes.

Padding & batching. A custom `collate_fn` dynamically pads each batch to its longest sequence and records true lengths. We use `pack_padded_sequence`: sequences are sorted by descending length, packed before the LSTM, and unsorted afterward. This guarantees the LSTM never processes PAD time steps.

Architecture. The model has approximately 5.8M parameters, structured as follows:

Table 1: LSTM classifier architecture

Layer	Specification	Output
Embedding	`vocab_size` $\times$ 128; `padding_idx=0`	128
BiLSTM	1 layer, hidden=256, bidirectional	512
Dropout	$p = 0.2$	512
Linear	$512 \to 4$	4 logits

Pooling strategy — last hidden state (length-aware). Since `pack_padded_sequence` ensures the LSTM stops at each sequence's last real token, `h_n` already holds the correct final states. We concatenate the forward and backward final hidden states into a 512-dim vector. We additionally explore masked mean and max pooling as architectural variants in Section 4, where the final representation is computed by averaging or taking the element-wise maximum over real token positions only, ignoring PAD states.

3.3 Transformer Pipeline

In the case of the transformer, the pretrained tokenizer chosen was DistilBERT, with 66,956,548 parameters. It is worth noting that word-level tokenization treats whole words as units, and maps unknown words to the unknown token. Transformers usually have additional special tokens. For example, in the case of BERT, the token [SEP] marks the end of a sequence, and the token [CLS] appears are the beginning of a phrase when the task is classification and the model should learn the distribution over the whole sequence.

```
Original : This is an example for Applied
    Machine Learning.
Task 1 (word-level nltk) : ['this', 'is',
    'an', 'example', 'for', 'applied',
    'machine', 'learning', '.']
  count  : 9

Task 2 (DistilBERT subword): ['this', 'is',
    'an', 'example', 'for', 'applied',
    'machine', 'learning', '.']
  count  : 9  (+2 for [CLS] and [SEP])
  input_ids     : [101, 2023, 2003, 2019,
     2742, 2005, 4162, 3698, 4083, 1012, 102]
  attention_mask: [1, 1, 1, 1, 1, 1, 1, 1, 1,
     1, 1]

Harder example: "The backpropagation
    algorithm optimizes transformer
    hyperparameters."
  Task 1 : ['the', 'backpropagation',
     'algorithm', 'optimizes', 'transformer',
     'hyperparameters', '.']
  Task 2 : ['the', 'back', '##pro', '##pa',
     '##gation', 'algorithm', 'opt', '##imi',
     '##zes', 'transform', '##er', 'hyper',
     '##para', '##meter', '##s', '.']
```

As shown in the examples above, for the first sentence, the word-level tokenizer produces 9 tokens, and the Transformer tokenizer produces the same visible token sequence. However, the Transformer input additionally includes the special tokens [CLS] at the beginning and [SEP] at the end, resulting in a total of 11 tokens. As mentioned above, these special tokens are required by the model: [CLS] is used to aggregate sequence-level information (e.g., for classification), while [SEP] marks the end of the input.

In the harder example above, the difference between tokenization strategies becomes more apparent. The word-level tokenizer treats each word as a single unit, whereas the Transformer tokenizer splits several complex words into multiple subword units (e.g., backpropagation, optimizes, and hyperparameters). This happens because the Transformer uses a subword tokenization method, which decomposes rare or morphologically complex words into smaller, more frequent components.

As illustrated, common words remain unchanged, while less frequent words are segmented into subwords with the ## prefix indicating continuation tokens. This approach allows the model to handle unseen words more effectively and reduces the size of the vocabulary, im-

proving generalization.

In the case of the whole dataset, each input sequence was padded or truncated to a fixed maximum length of 128 tokens to ensure uniform input size across batches. Moreover, an attention mask was generated for each sequence, where real tokens are marked with 1 and padding tokens with 0, allowing the model to ignore padded positions during attention computation. We fine-tuned the pretrained distilbert-base-uncased model with a sequence classification head, configured to output predictions over the 4 AG News classes.

3.4 Training Setup

LSTM. We train with cross-entropy loss using the Adam optimizer (lr = 10^{-3}), batch size 64, for 6 epochs. Gradient norms are clipped to 1.0 at each step. Training and validation loss/accuracy are recorded per epoch. Average training time was 28.3s per epoch (170s total).

Table 2: Task 1 hyperparameters

Max seq length	128	Optimizer	Adam
Embedding / Hidden dim	128 / 256	Learning rate	10^{-3}
LSTM layers / Bidirectional	1 / Yes	Batch size	64
Dropout	0.2	Epochs	6
Loss	CrossEntropyLoss	Gradient clipping	max norm 1.0

Transformer. For this case, the optimizer used to fine-tune was AdamW with a learning rate of 2×10^{-5}, as this configuration is usually used to train transformers, it allows for stable fine-tuning of pre-trained transformers, and it has the advantage of incorporating weight decay for regularization.

Furthermore, the batch size was 16, so as to balance computational efficiency and memory constraints on GPU. The model was trained for 3 epochs, which we found sufficient for convergence without overfitting. To reduce the chance of this happening, a weight decay of 0.05 was applied to reduce overfitting by penalizing large weights. Finally, mixed precision training (fp16) was optionally enabled to accelerate training on GPU without significantly affecting performance.

Table 3: Task 2 hyperparameters

Model	DistilBERT	Optimizer	AdamW
Max seq length	128	Learning rate	2×10^{-5}
Tokenizer	Subword (pretrained)	Batch size	16
Padding / Truncation	Max length	Epochs	3
Weight decay	0.05	Mixed precision	fp16
Loss	CrossEntropyLoss	Scheduler	None

4 Results and Analysis

4.1 Performance

The baseline BiLSTM achieved a best validation accuracy of 91.66% (epoch 3, based on lowest validation loss) and a final validation accuracy of 91.75% after 6 epochs. Test accuracy was 91.39%. After the full ablation study, the best LSTM variant (LSTM-Bi-Max with GloVe embeddings) reached a test accuracy of 93.17% and a macro F1 of 93.16%.

The fine-tuned DistilBERT reached a best validation accuracy of 94.8% at epoch 3 and a test accuracy of 94.58%, with a macro F1 of 94.59%. Across all Transformer training strategies, full fine-tuning remained the strongest configuration.

Extended experiments are reported in Appendix A. For Task 1, we performed an ablation study over 7 architectural variants (Table 9) and compared GloVe against random embedding initialisation (Table 10), with the best configuration (LSTM-Bi-Max + GloVe) reaching 93.17% test accuracy. For Task 2, we compared three fine-tuning strategies: full fine-tuning, frozen encoder, and LoRA (Table 11) — and conducted an $L_{\max}$ ablation over $\{32, 64, 128\}$ tokens (Table 12).

4.2 Training Behavior

LSTM. Table 4 shows the per-epoch learning curves for the baseline BiLSTM. Training loss decreased steadily across all 6 epochs, reaching 0.024 by the final epoch, indicating strong convergence. Validation loss improved sharply in the first 3 epochs (from 0.313 to 0.251) and then increased monotonically from epoch 4 onward, reaching 0.365 at epoch 6 — a clear sign of overfitting. The best checkpoint, restored by early stopping, corresponds to epoch 3 (lowest validation loss = 0.251). Validation accuracy tracked validation loss, peaking at 91.7% and then plateauing. The widening gap between training loss (0.024) and validation loss (0.365) at epoch 6 confirms that dropout and gradient clipping provide only partial regularization for this dataset size.

Table 4: Baseline BiLSTM per-epoch learning curves (Colab T4 GPU)

Epoch	Train Loss	Val Loss	Val Acc	Time (s)
1	0.4627	0.3130	0.8878	29.3
2	0.2288	0.2532	0.9133	27.3
3	0.1530	0.2513	0.9166	27.2 **[best]**
4	0.0969	0.2731	0.9155	29.3
5	0.0578	0.3138	0.9165	26.8
6	0.0236	0.3650	0.9175	30.1

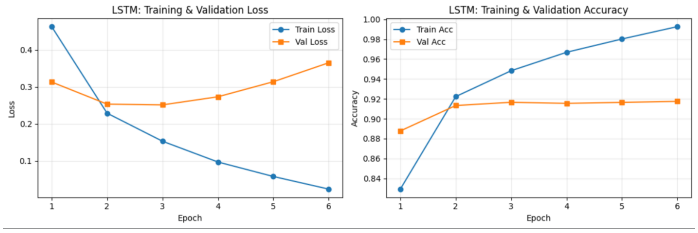


Figure 1: LSTM learning curves: training loss vs. epoch (left) and validation accuracy vs. epoch (right).

Table 5: Baseline DistilBERT per-epoch learning curves (Colab T4 GPU)

Epoch	Train Loss	Val Loss	Val Acc	Time (s)
1	0.2349	0.1918	0.9387	451
2	0.1421	0.1827	0.9466	451 **[best]**
3	0.0974	0.2117	0.9480	451

Transformer. Table 5 shows the per-epoch learning curves for the baseline full fine-tuning run. Training loss decreased steadily from 0.235 to 0.097 across 3 epochs. Validation loss improved from epoch 1 to epoch 2 (0.192 → 0.183) and then rose slightly at epoch 3 (0.212), suggesting mild overfitting in the final epoch. Despite this, validation accuracy increased monotonically across all 3 epochs, reaching 94.8% at epoch 3. This behaviour differs markedly from the LSTM: the Transformer's pretrained representations allow fast convergence with only minor generalization degradation.

4.3 Comparison Table

Table 6 summarises the key quantitative differences between the best LSTM configuration and the best Transformer configuration. The Transformer consistently outperforms the LSTM in both validation and test accuracy, achieving a gain of approximately 1.3–3.5 percentage points depending on the LSTM variant. This improvement comes at the cost of a significantly larger model (∼67M vs ∼5.8M parameters) and considerably longer training time per epoch (∼451s vs ∼28s on the same T4 GPU). The LSTM, however, trains from scratch on domain data and requires no pretrained checkpoint.

4.4 Qualitative Error Analysis

LSTM. Full misclassified examples are listed in Appendix B.1. The dominant pattern is **Business ↔ Sci/Tech**: technology company articles framed with business vocabulary (and vice versa) consistently confuse the model. Economic news with global framing also produces World → Business errors. Named entities below the frequency threshold are mapped to UNK, removing disambiguating cues.

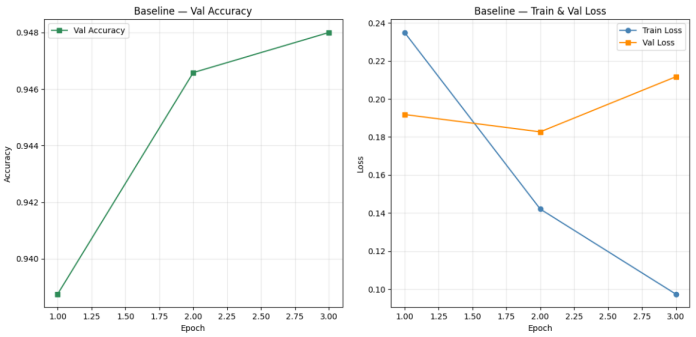


Figure 2: Transformer learning curves: validation accuracy vs. epoch (right), training loss and validation loss vs epoch (left)

Table 6: Direct comparison: LSTM vs. Transformer

Metric	LSTM (best)	DistilBERT (full FT)
Model	LSTM-Bi-Max + GloVe	distilbert-base-uncased
Validation accuracy (best)	93.7%	94.8%
Test accuracy	93.2%	94.6%
Macro F1	93.2%	94.6%
Parameters (total)	∼4.7M	∼66.9M
Trainable parameters	∼4.7M	∼66.9M
Training time / epoch	∼38s (T4 GPU)	∼451s (T4 GPU)
Total training time	∼153s	∼1353s
Epochs to convergence	4	3
Environment	Google Colab T4 GPU	

Table 7: LSTM per-class classification report (test set)

Class	Prec.	Rec.	F1	Supp.
World	0.92	0.92	0.92	1,900
Sports	0.95	0.97	0.96	1,900
Business	0.91	0.86	0.88	1,900
Sci/Tech	0.87	0.90	0.89	1,900
Macro avg	0.91	0.91	0.91	7,600

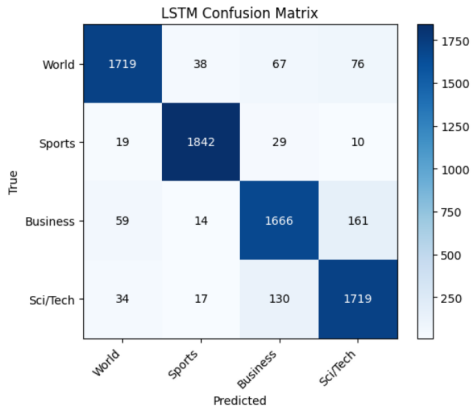


Figure 3: LSTM confusion matrix on the test set.

Transformer. Full misclassified examples are listed

in Appendix B.2. The same **Business ↔ Sci/Tech** pattern dominates: IBM and Google articles labelled Sci/Tech are predicted as Business due to financial framing. World → Business errors arise when geopolitical articles involve commodity prices (oil, Venezuela). One outlier is Bush's missile defence article (World → Sci/Tech), where technical vocabulary overrides the geopolitical context. Sports remains the cleanest class in both models.

Table 8: Transformer per-class classification report (test set)

Class	Prec.	Rec.	F1	Supp.
World	0.93	0.94	0.93	1,900
Sports	0.97	0.98	0.97	1,900
Business	0.92	0.89	0.90	1,900
Sci/Tech	0.89	0.91	0.90	1,900
Macro avg	0.93	0.93	0.93	7,600

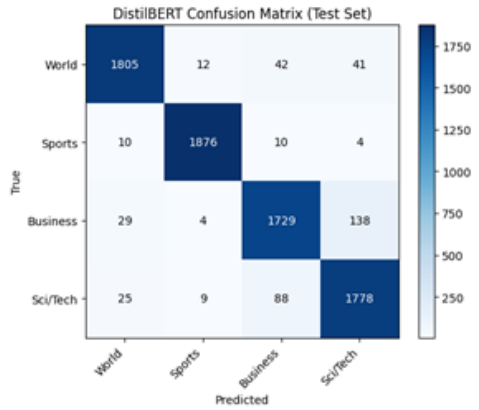


Figure 4: Transformer confusion matrix on the test set.

5 Discussion and Conclusion

The main hardships of the LSTM were related to rare words, which are mapped to UNK tokens, resulting in a loss of key semantic signals. We observed higher confusion between semantically overlapping classes (World↔Business, Business↔Sci/Tech), with Business being a particularly complex category. Additionally, as expected from theory, LSTMs show limited ability to capture long-range dependencies compared to attention-based models. Since the model learned representations from scratch, it also suffered from higher data inefficiency.

The Transformer achieved higher accuracy (∼94.6%) due to pretrained contextual representations, and handled ambiguous or context-dependent inputs more robustly. Global self-attention reduced confusion across overlapping classes, as the model could attend to the full sequence rather than relying on a single fixed-size hidden state.

Nonetheless, the LSTM architecture is considerably simpler and faster to train (∼38s vs ∼451s per epoch on the same T4 GPU), which is a major practical advantage. It is also easier to debug and interpret, since the full pipeline — tokenization, vocabulary, padding — is explicit and user-controlled.

Regarding tokenization, word-level tokenization is simpler and more interpretable, but suffers from OOV problems by mapping unknown words to UNK, limiting generalization to rare or morphologically complex words. Subword tokenization handles unseen words by decomposing them into known pieces, improving robustness and reducing vocabulary size while increasing coverage, though at higher computational cost and longer sequence lengths.

Implementation-wise, LSTMs require custom pipelines covering tokenization, vocabulary construction, padding, and packing, while Transformers rely on well-maintained libraries that abstract these steps, though at the cost of conceptual complexity. Memory requirements also differ significantly: LSTMs are lightweight enough for CPU training, whereas Transformers require GPU acceleration for efficient fine-tuning. One concrete improvement we would explore next is replacing DistilBERT with RoBERTa-base, which uses a more robust pretraining procedure and has consistently outperformed BERT-style models on classification benchmarks, potentially closing the remaining 1–2% accuracy gap.

6 Statement of Contributions

Bora Sanal built the LSTM classifier pipeline, including tokenization, vocabulary construction, model architecture, training and test evaluation with error analysis. Andrea Gomez built the transformer pipeline, model architecture, training and test evaluation with error analysis. She also contributed to the introduction and wrote the discussion section. Hadi was responsible for the "Going Beyond" aspect of the assignment. He performed an ablation study over recurrent architecture, directionality, and pooling for the LSTM-based models, comparing a total of seven variations. He also compared random initialization against GloVe initialization. For the Transformer, he compared full fine-tuning, a frozen encoder, and LoRA, and additionally conducted an ablation study on the maximum sequence length to analyze

the trade-off between accuracy and training efficiency. He analyzed the results of all these experiments.

7 Statement on the Use of LLMs and Other Resources

Large Language Model (ChatGPT) and AI-based writing assistant (Grammarly) were used to improve the clarity, grammar, and academic tone of the written content. ChatGPT was also consulted to help identify, iterate and fix bugs during the code development process. No code or data was shared with other teams.

A Going Beyond: Extended Experiments

A.1 Task 1 — Ablation Study

To understand the contribution of each architectural choice, we trained seven variants of the BiLSTM classifier, varying directionality (unidirectional vs. bidirectional), pooling strategy (last hidden state, masked mean, masked max), and recurrent unit type (LSTM vs. GRU). All variants used identical hyperparameters. Results are shown in Table 9.

Table 9: Ablation study results (all variants, Colab T4 GPU)

Model	#Params	Epochs	Val Acc	Test Acc	Macro F1	Time/ep
LSTM-Bi-Max	5,821,188	4	0.9262	**0.9184**	**0.9181**	38.9s
LSTM-Uni-Last	5,424,900	5	0.9206	0.9132	0.9128	19.0s
LSTM-Bi-Mean	5,821,188	4	0.9205	0.9125	0.9123	38.8s
GRU-Bi-Mean	5,623,556	4	0.9203	0.9117	0.9117	38.6s
GRU-Uni-Mean	5,326,084	4	0.9157	0.9099	0.9098	25.7s
LSTM-Bi-Last	5,821,188	5	0.9167	0.9092	0.9097	26.6s
LSTM-Uni-Mean	5,424,900	4	0.9202	0.9071	0.9068	26.8s

Bidirectional models consistently outperform their unidirectional counterparts (e.g., LSTM-Bi-Mean 91.25% vs. LSTM-Uni-Mean 90.71%), confirming that backward context is informative for news classification. Max pooling outperforms both mean and last-hidden pooling for bidirectional LSTMs, suggesting that the most salient token-level feature across the sequence is more discriminative than the average or the final state. GRU and LSTM perform comparably, with LSTM-Bi-Max achieving the best test accuracy overall. Learning curves are shown in Figures 7 and 8 in Appendix C.

A.2 Task 1 — GloVe vs. Random Embedding Initialisation

We compared random embedding initialisation against GloVe 6B 100-d vectors for the best ablation configuration (LSTM-Bi-Max, d_emb=100). Both models were fine-tuned end-to-end. Results are shown in Table 10.

Table 10: GloVe vs. random initialisation (LSTM-Bi-Max, d_emb=100)

Init	Val Acc	Test Acc	Macro F1	Time/ep
Random (100d)	0.9237	0.9200	0.9199	38.5s
GloVe (100d)	**0.9374**	**0.9317**	**0.9316**	38.3s

GloVe initialisation improves test accuracy by 1.17 percentage points (92.00% → 93.17%) with no additional training cost. The GloVe model also converges faster: it reaches its best validation loss at epoch 2 compared to epoch 3 for random initialisation, suggesting that pre-trained embeddings provide a stronger starting point. Learning curves are shown in Figures 9 and 10 in Appendix C.

A.3 Task 2 — Training Strategy Comparison

We compared three fine-tuning strategies for DistilBERT, keeping all other hyperparameters fixed. Full fine-tuning updates all 66.9M parameters; frozen-encoder training updates only the classification head (593K parameters); LoRA injects low-rank adapter matrices into the attention layers (741K trainable parameters, rank $r = 8$).

Table 11: Training strategy comparison (DistilBERT, Colab T4 GPU)

Strategy	Trainable	Val Acc	Test Acc	Macro F1	Time/ep
Full Fine-tuning	66,956,548	**0.9474**	**0.9457**	**0.9457**	457s
LoRA (r=8)	741,124	0.9183	0.9126	0.9126	295s
Frozen Encoder	593,668	0.8972	0.8966	0.8964	144s

Full fine-tuning achieves the highest accuracy but requires the most compute. Frozen-encoder training is fastest (144s/epoch, 3.2× speedup) but loses 4.9 percentage points of test accuracy, confirming that encoder representations need task-specific adaptation. LoRA recovers most of the accuracy gap (91.26% vs. 94.57%) while training only 1.1% of the parameters and running 35% faster per epoch. Learning curves are shown in Figures 13 and 14 in Appendix C.

A.4 Task 2 — Maximum Sequence Length Ablation

We studied the effect of the maximum input length $L_{\max} \in \{32, 64, 128\}$ on accuracy and training speed, using full fine-tuning for 1 epoch each.

Table 12: $L_{\max}$ ablation (DistilBERT, 1 epoch, full fine-tuning)

$L_{\max}$	Avg Tokens	Test Acc	Time/ep	Speedup
32	31.7	0.9287	343s	1.12×
64	49.9	0.9391	326s	1.18×
128	52.7	**0.9417**	384s	1.00×

Dropping from $L_{\max} = 128$ to 64 costs only 0.26 percentage points while providing an 18% speedup, consistent with the mean sequence length in AG News (∼44 tokens). Reducing to 32 causes a more noticeable drop (1.30 pp). For latency-sensitive deployments, $L_{\max} = 64$ represents a strong accuracy–efficiency trade-off.

B Misclassified Test Examples

B.1 LSTM Misclassified Examples

Table 13: LSTM misclassified examples (baseline BiLSTM)

Excerpt	True	Pred
British industry at best in 10 years: manufacturing enjoying strongest performance in almost 10 years...	Business	Sci/Tech
Nobel Economics Prize Awarded: Finn Kydland and Edward Prescott won the 2004 Nobel economics prize...	Business	Sci/Tech
Producer Price Surge Fuels Inflation Fears: prices surged 1.7% in October, sharpest rise in 15 years...	World	Business
Leading Indicators Down for 5th Month: key forecasting gauge of U.S. economic activity fell again...	Business	Sci/Tech
Make hotels just like home: guests making nonsense of room-pricing with Internet-aided discount-hunting...	Sci/Tech	Business
Plan B Proposes Its Own Alternatives: CEO of Plan B Technologies said the company itself is a "plan B"...	Sci/Tech	Business
Opel to cut payroll by 9,500: Opel gets by without layoffs despite headline suggesting otherwise...	Business	Sci/Tech
Worldwide PC Market Seen Doubling by 2010: number of PCs expected to reach 1.3 billion...	Sci/Tech	Business
Hurricane forecasters debate usefulness of "skinny line" in tracking maps...	Business	Sci/Tech
Gateway Reports Smaller Quarterly Loss while restructuring and integrating eMachines acquisition...	Sci/Tech	Business

B.2 Transformer Misclassified Examples

Table 14: Transformer misclassified examples (full fine-tuning)

Excerpt	True	Pred
IBM to hire even more new workers: computing giant plans biggest headcount since 1991...	Sci/Tech	Business
Some People Not Eligible to Get in on Google IPO: billed as a way for everyday people to participate...	Sci/Tech	Business
Venezuela Prepares for Chavez Recall Vote: defeat could produce turmoil in world oil market...	World	Business
Intel to delay product aimed for high-definition TVs: postponed launch of video display chip...	Business	Sci/Tech
Yahoo! Ups Ante for Small Businesses: price cuts on domain registration and Web hosting products...	Business	Sci/Tech
IBM Buys Two Danish Services Firms to broaden presence in Scandinavia...	Sci/Tech	Business
Oil prices bubble to record high: light sweet crude closed at a record overnight...	World	Business
Bush Promotes His Plan for Missile Defense System: opponents putting national security at risk...	World	Sci/Tech
Ross Stores Profit Plummets 40% due to problems with a new computer system...	Business	Sci/Tech
Intuit Posts Wider Loss After Charge: maker of TurboTax posted wider quarterly loss...	Sci/Tech	Business

C Learning Curves and Confusion Matrices

C.1 Task 1 — LSTM Baseline

C.2 Task 1 — Ablation Study

C.3 Task 1 — GloVe vs. Random Initialisation

C.4 Task 2 — Transformer Baseline

C.5 Task 2 — Training Strategy Comparison

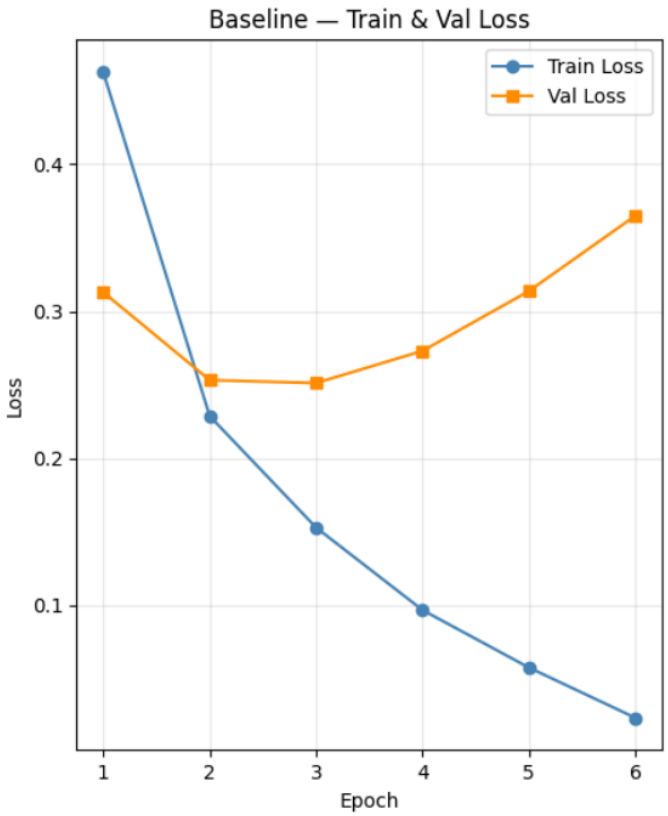


Figure 5: LSTM Baseline: training loss and validation loss vs. epoch.

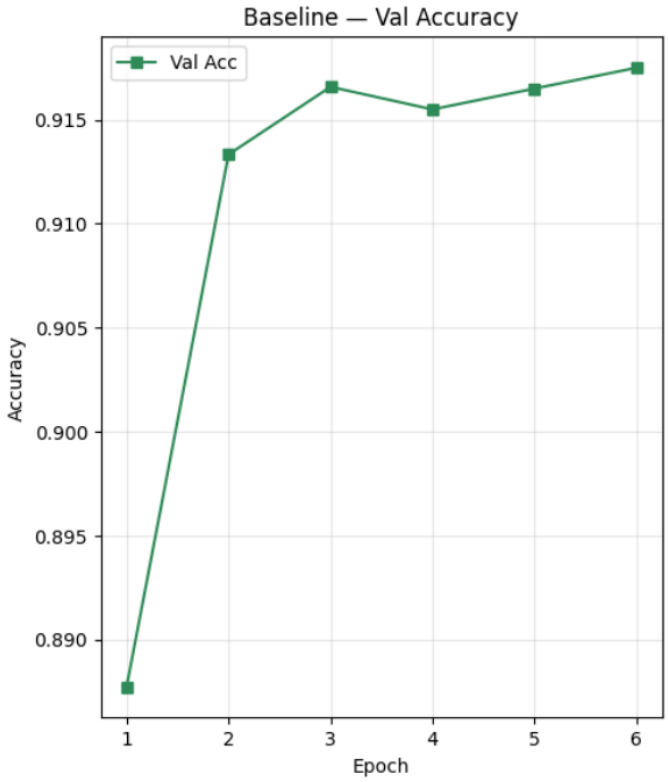


Figure 6: LSTM Baseline: validation accuracy vs. epoch.

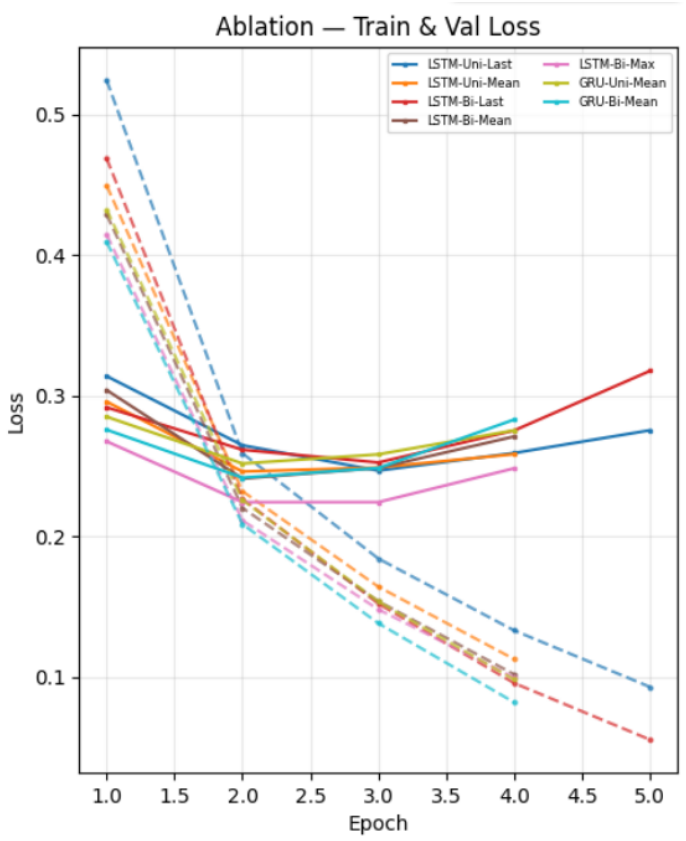


Figure 7: Ablation study: training loss (dashed) and validation loss (solid) vs. epoch for all 7 model variants.

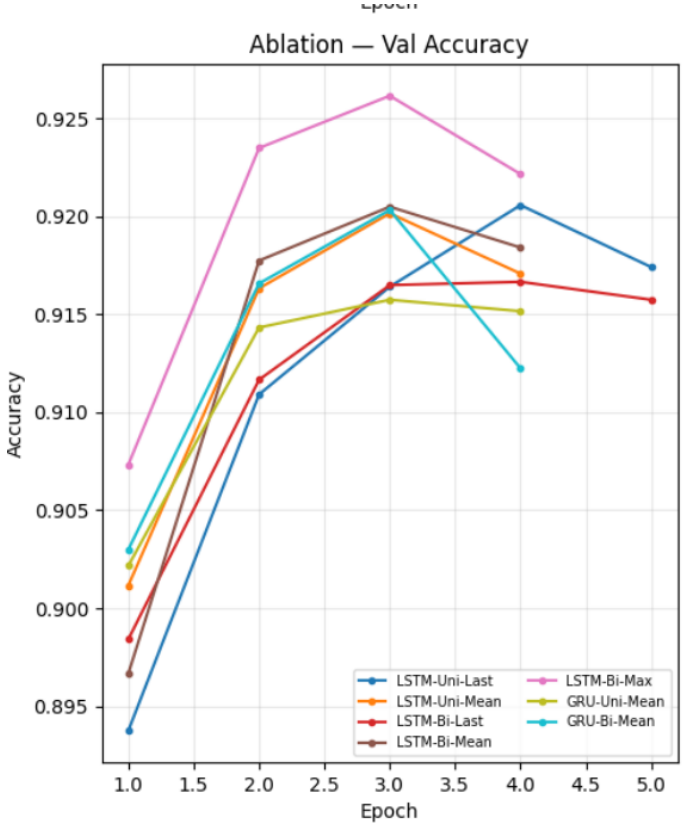


Figure 8: Ablation study: validation accuracy vs. epoch for all 7 variants.

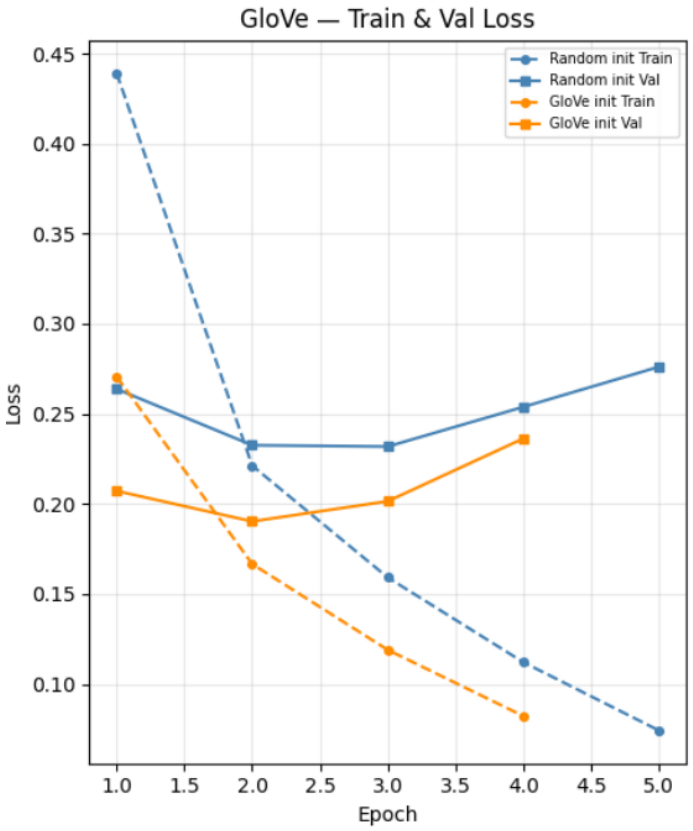


Figure 9: GloVe: training loss (dashed) and validation loss (solid) vs. epoch for random and GloVe initialisations.

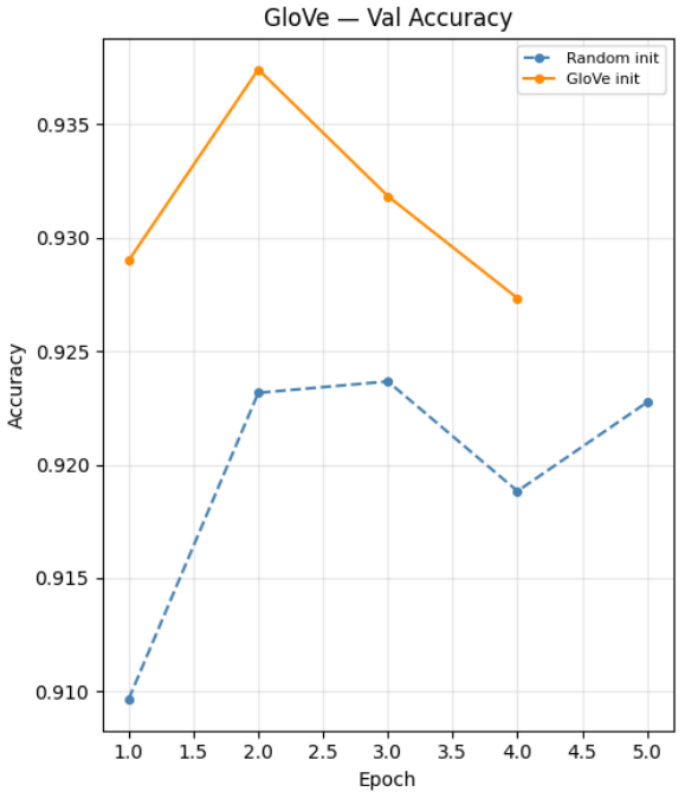


Figure 10: GloVe: validation accuracy vs. epoch for random and GloVe initialisations.

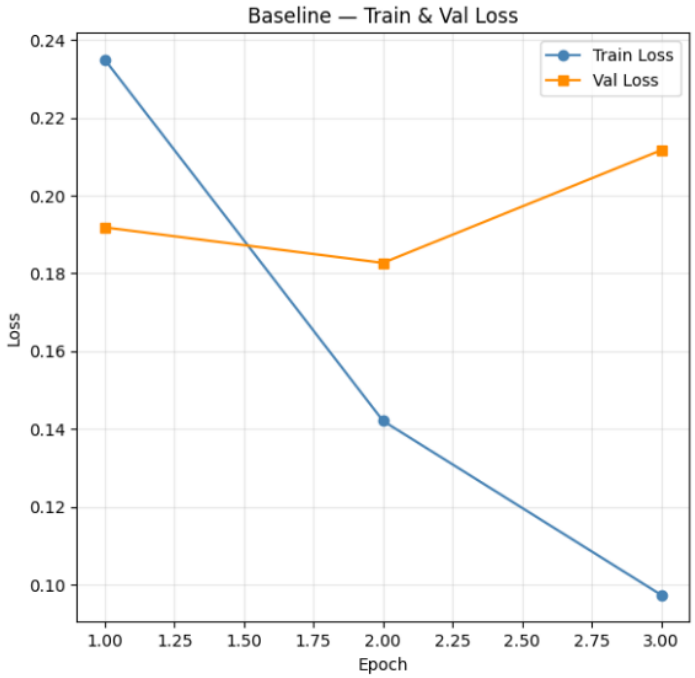


Figure 11: Transformer Baseline: training loss and validation loss vs. epoch (full fine-tuning, lr=2×10^{-5}, 3 epochs).

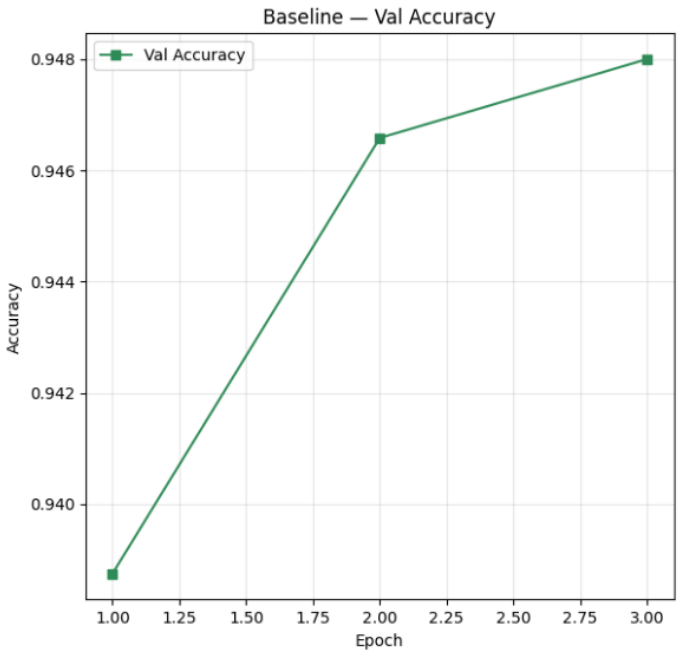


Figure 12: Transformer Baseline: validation accuracy vs. epoch.

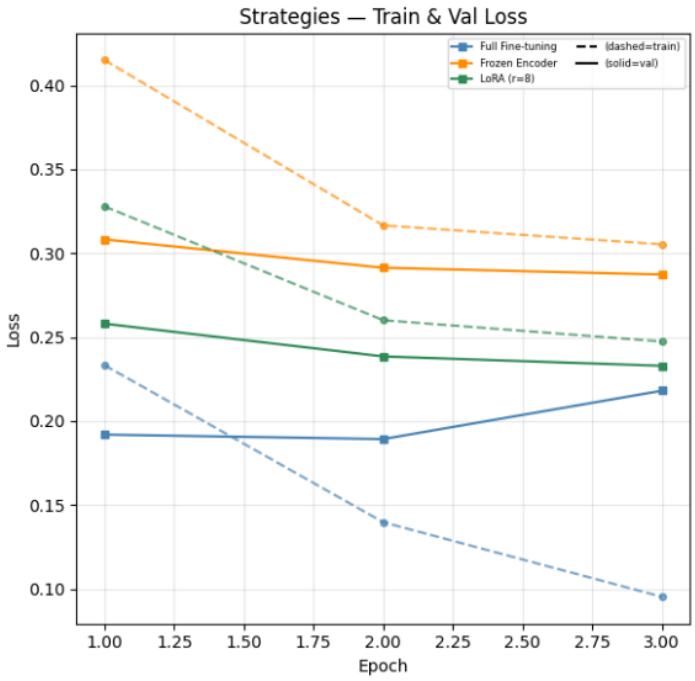


Figure 13: Strategy comparison: training loss (dashed) and validation loss (solid) vs. epoch for full fine-tuning, frozen encoder, and LoRA (r=8).

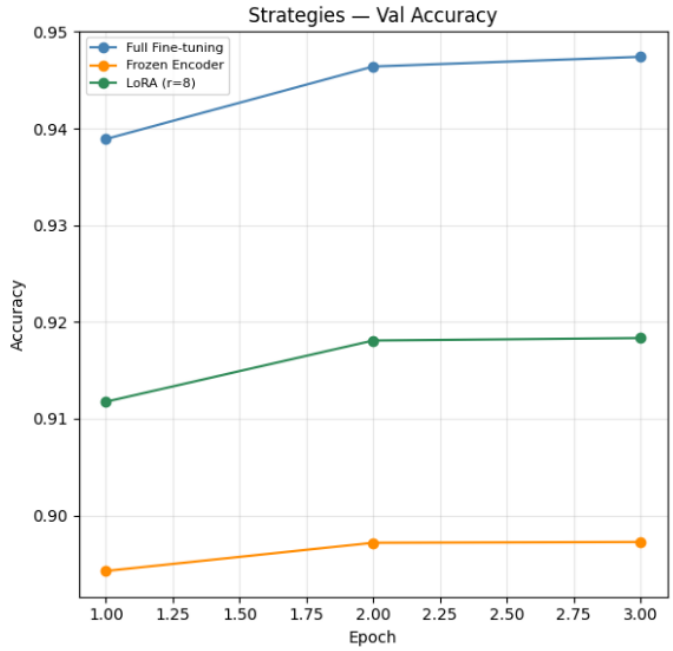


Figure 14: Strategy comparison: validation accuracy vs. epoch for all three training strategies.